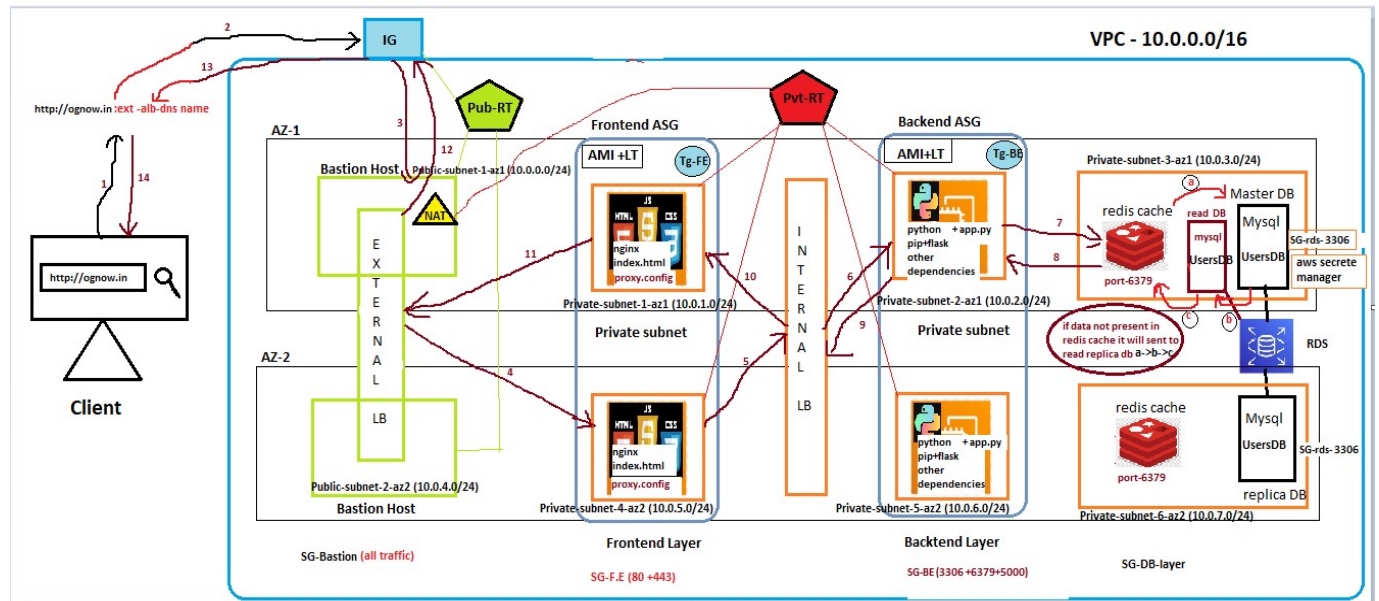


Deploy a 3 tier Fullstack app with python backend + RDS cache



Prerequisite

1. VPC (10.0.0.0/16)+IG+NAT
2. 2 availability zone (AZ-1 +AZ-2)
3. Create total 8 subnets (4 for each az)
 - a. Az1 (1 public +3private)
 - b. Az2 (1 public +3 private)
4. 2 bastion host + 2 pvt frontend server + 2 pvt backend server)
5. RDS users DB +(1 RDS master in az1 + 1 RDS replica in az2
6. Public-RT + Private-RT
7. 1 FE-ASG (AMI + LT) + 1 BE-ASG (AMI +LT).
8. 1 Internal ALB (BE-Tg1)+ 1 External ALB (FE-Tg1).
9. AWS secrets manager + IAM role
10. Get all required resources from – <https://github.com/itmukesh-16/python-backend-testing.git>

Az1

Public → 10.0.0.0/24 Public-subnet-1-az1
 Frontend → 10.0.1.0/24 Private-subnet-1-az1
 Backend → 10.0.2.0/24 Private-subnet-2-az1
 DB → 10.0.3.0/24 Private-subnet-3-az1

Az2

Public → 10.0.4.0/24 Public-subnet-1-az2
 Frontend → 10.0.5.0/24 Private-subnet-4-az2
 Backend → 10.0.6.0/24 Private-subnet-5-az2
 DB → 10.0.7.0/24 Private-subnet-6-az2

Implementation

Infrastructure & Network Set up-

1. Create VPC (10.0.0.0/16) and attach IG and NAT
2. Connect Public-RT to IG and Private-RT to NAT (**Make sure the NAT connected to one of the public subnet for internet supply and while creating select Regional option**)
3. Connect Public-RT → Public subnet , Private-RT → Private subnets.
4. Create 4 SGs, as of now give all access from anywhere for all SGs (SG-Bastion , SG-FE , SG-BE , SG -DB) .later we can open required port only

[create initially one bastion host , check IG connection and deploy 1 private server , check NAT connection] once success fully tested the connection go to the next phase.

RDS database set up -

5. Create sone RDS subnet group .
6. Deploy 1 RDS DB named usersDB in az1 private subnet (Enable aws managed secrete keys)
7. Test connection to RDS DB using mysql workbench via Bastion Host.(mysql workbench → bastion host → RDS DB) .
8. Now create 1 replica usersDB in az1 +1 replica usersDB in az2 (test connection for read DB deployed in both AZs.)
9. Create a redis cache for Masterdb (usersDB).**make sure the SG attached to cache db opens port 6397**

Backend setup –

1. Deploy Backend-az1 server in private-backend-subnet ,check internet connectivity
2. Connect Frontend server in az1 and switch to root user
3. install [python+pip+mariadb105]
4. Create a directory **backend** -> then create backend.py ,requirements.txt , usersdb.sql .
5. **requirements.txt** {flask + flask-cors + mysql-connector-python + redis + cryptography + pymysql+boto3}
6. under usersdb.sql give users table schema.
7. Using **mariadb105** (mysql agent) test connection to RDS usersdb .(for private server to RDS usersdb)

[at the RDS time, if RDS credentials – self managed]

To execute sql query by staying in the backend private server

```
#mysql -h <endpoint dns> -u admin -p'dbpassword' < usersdb.sql
```

To manage rds usersdatabase in the backend

```
#mysql -h <endpoint dns> -u admin -p
```

[if the RDS creation time , RDS credentials managed by – aws secrete manager , then we need to create an IAM role with permission (SecretsManagerReadWrite) and add it to backend server .otherwise it wont allow to acces]

8. Include backend code in **backend.py** then run **#python3 backend.py**
9. For temporary fetch the result , open new terminal then check process(ps aux | grep python) and port(ss -tln) ,whether the python service is running or not if running then fetch user data from usersdb by –

```
10. curl -X GET http://localhost:5000/users
```

if there is no out put or you get request : *timed out* ,it means EC2 is not able to connect to the cache database .(check if SG of redis cache allowing 6379 , subnet , VPC)

11. to effieciently handle the process in the same server ,you can configure #pm2 for prcess to run.
12. Then #systemctl enable python3 backend.py

Frontend Set up-

13. Create Frontend server in az1 , connect and switch to root
14. Configure (nginx +index.html +proxy.conf)
15. Install nginx and replace frontend code in **/usr/share/nginx/html/index.html**
16. Configure index.html , make sure the front end code satisfy the structure which will present the data fetched from the backend.otherwiser it wont show in browser
17. Configure proxy.conf file to support reverse –proxy ie redirect the request from external ALB → frontend server → internal ALB.

```
18. #yum install nginx -y
19. #sudo vi /etc/nginx/conf.d/reverse-proxy.conf
20. #sudo nginx -t
21. #sudo systemctl reload nginx
```

22. Now run → #systemctl start nginx
23. Again →#systemctl enable nginx
24. For testing ,try to fetch the data from backend

```
# curl -X GET http://backend-server-private-ip:5000/users
```

If its fetching data it means it successful.api and index page is working

Create AMI and Launch Template –

[Before creating LT , make sure the Frontend and backend services are running ie nginx , python . also make sure you configured the services in such a way it will autostart the services everytime there is a reboot , deploy new instance,system chrash or new session

For ex nginx `#systemctl enable nginx`]

25. Select Frontend-az1 server >AMI>Launch Template (select desired vpc,subnets azs for deployment)>Frontend-LT
26. Using Frontend-LT deploy another frontend server in **Private-subnet-4-az2**.
27. Then do for Backend-az1 server>AMI>Launch Template>Backend-LT
28. Using Backend-LT deploy another frontend server in **Private-subnet-4-az2**.

Configure Load Balancer

Internal load balancer for Backend server HA:

- a. Create a target group BE-Tg1(select the two backend server from both AZs)
- b. Give path '/', http(80) protocol and port :5000 (cause python flask running on port 5000) and it will check the port health to distribute the traffic.
- c. Now create an application load balancer(ALB) **internal-facing** configure port 5000.
- d. Add target group **BE-Tg1** and create internal load balancer **internal-dns**.
- e. Now for test fetch the data using **internal-alb-dns** from frontend server.

curl -X GET <http://internal-alb-dns:5000/users>

- If the test successful.then update the **internal-alb-dns** link in **revers_proxy.conf** file of the both frontend servers.
- Otherwise the frontend will always point to fixed private backend server ip ,as a result the load balancer will not work properly
- Also update **Frontend-LT** or else it will contain the old server configuration and will cause issue in ASG .

```
server {
    listen 80;
    server_name _;

    # Proxy users endpoints directly
    location /users {
        proxy_pass http://internal-alb-dns:5000;
        proxy_http_version 1.1;
    }
}
```

External/Internet load balancer for Frontend server HA:

- Create a target group FE-Tg1(select the two backend server from both AZs)
- Give path '/', http protocol and port :80 (cause nginx running on port 80) and it will check the port health to distribute the traffic.
- Now create an application load balancer(ALB) **External-facing** configure port 80.
- Add target group **FE-Tg1** and create External load balancer **Ext-alb**.
- Now access the **External-alb-dns** in browser and see if the the index page is properly working or not .

Now create ASG-

[make sure the Launch Template you providing to ASG has latest/updated configuration of running instances ,otherwise it will not deploy the updated instance configuration and services running inside the servers . As a result , it might not give expected result]

- Configure Frontend -ASG for Frontend server HA
- Configure Backend – ASG for backend server HA

For testing purpose lower the cpu utilisation threshold and see if the autoscalling is working or not.

Now configure SGs-

Since we had opened all the ports for all the security groups for testing and its not recommended .it may cause security breach .

Hence only open required inbound and outbound ports for all the SGs

#	Security Group	Attach to	Required Inbound	Required Outbound	
1	SG-Bastion	Bastion Host	22 ← Your IP	22 → SG-FE , SG-BE	
2	SG-ALB-FE	Internet-facing ALB	80,443 ← Internet	80 → SG-FE	
3	SG-FE	Frontend ASG	80 ← SG-ALB-FE 22 ← SG-Bastion	80 → SG-Internal-ALB 443 → Internet	
4	SG-Internal-ALB	Internal ALB	80 ← SG-FE	5000 → SG-BE	
5	SG-BE	Backend ASG	5000 ← SG-Internal-ALB 22 ← SG-Bastion	3306 → SG-RDS 6379 → SG-Redis 443 → Internet	
6	SG-Redis	ElastiCache Redis	6379 ← SG-BE	No application outbound required	
7	SG-RDS	RDS MySQL	3306 ← SG-BE	No application outbound required	

Task Over

